PROYECTO FIN DE GRADO

Yummy!

2º DAM

Tutor: Gonzalo Pérez Crespo

Marina Maestre Sánchez

2025/2026

Tabla de contenido

Tabla de contenido 1
1. Resumen. 3
2. Introducción. 3
2.1. Contexto y motivación. 3
2.2. Enfoque en accesibilidad y diseño inclusivo. 3
2.3. Objetivos. 3
2.4. Alcance y limitaciones. 4
3. Análisis y planificación. 4
3.1. Necesidad detectada. 4
3.2. Relación con proyectos tipo. 4
3.3. Contexto específico: accesibilidad y valor personal 5
3.4. Requisitos funcionales. 5
3.5. Requisitos no funcionales. 6
3.6. Análisis de Usuarios y Roles. 6
3.7. Casos de Uso. 7
3.8. Diagramas UML. 7
4. Desarrollo e Implementación. 9
4.1. Configuración del Entorno de Desarrollo y Control de Versiones. 9
4.2. Herramientas y Entorno de Ejecución. 9
4.3. Gestión del Repositorio en GitHub. 9
4.4. Estructura de Directorios e Inicialización. 10
4.5. Configuración del Backend y Dependencias. 12
4.6. Configuración de Variables de Entorno. 13
4.7. Arquitectura del Backend y Configuración del Servidor Express. 13
4.8. Módulo de Conexión a la Base de Datos 14
4.9. Lógica de autenticación y cifrado 15
4.10. Validación del Sistema con Postman. 16
4.11. Modelado de datos para el sistema de recetas. 17
4.12. Lógica de gestión de recetas. 18
4.13. Definición de rutas y protección de recursos. 19
4.14. Validación final del sistema de autenticación y seguridad 19
4.15. Conclusión de la arquitectura backend y éxito de persistencia. 20
5. Bibliografía y Webgrafía. 20

1. Resumen.

"Yummy! – Gestor de Recetas" es un Proyecto Intermodular de 2ºDAM orientado al desarrollo de una aplicación web full-stack para la gestión de recetas personales. Su función principal es permitir al usuario crear, consultar, modificar y eliminar recetas de manera sencilla y organizada, preservando así un recetario digital accesible desde cualquier dispositivo.

El proyecto combina dos dimensiones: la parte técnica, centrada en la construcción de un sistema CRUD utilizando (Node.js, s.f.), Express y una base de datos NoSQL; y la parte personal, que busca conservar la herencia culinaria andaluza y marroquí que forma parte de mi historia familiar. La accesibilidad es un pilar esencial del diseño, con un enfoque centrado en crear una interfaz clara, legible y adaptable a usuarios con distintos niveles de experiencia digital.

2. Introducción.

2.1. Contexto y motivación.

Yummy! nace de algo muy cotidiano: la dificultad de mantener las recetas organizadas cuando están repartidas entre libretas, notas del móvil, fotos o recuerdos. Esta problemática, tan común y práctica, se une en mi caso a un componente profundamente personal: la mezcla cultural que define mi cocina, con raíces andaluzas y marroquíes.

Una parte importante de este proyecto es precisamente la intención de preservar ese legado gastronómico familiar y convertirlo en un espacio digital accesible y cuidado. Pero Yummy! también es el cierre de mi formación técnica en el ciclo de Desarrollo de Aplicaciones Multiplataforma, donde he querido integrar todo lo aprendido en programación, bases de datos, desarrollo web y diseño de interfaces.

2.2. Enfoque en accesibilidad y diseño inclusivo.

La accesibilidad no es un añadido, sino uno de los pilares del proyecto. Mi objetivo es que cualquier persona pueda utilizar la aplicación con comodidad, independientemente de su edad, sus capacidades o su experiencia con la tecnología. Esto incluye trabajar con alto contraste, tipografías legibles, estructuras semánticas correctas y una navegación especialmente clara.

Este enfoque tiene una doble intención: cuidar tanto la experiencia del usuario como la calidad técnica del proyecto, demostrando que el diseño inclusivo es compatible con un desarrollo profesional.

2.3. Objetivos.

Objetivo general:

- Desarrollar una aplicación web full-stack para la gestión de recetas personales mediante un sistema CRUD completo y funcional.

Objetivos específicos:

- Diseñar una interfaz accesible, clara y usable.
- Implementar una API REST con Node.js y Express.
- Definir y modelar una base de datos NoSQL.
- Aplicar buenas prácticas de UX/UI y accesibilidad digital.
- Controlar el desarrollo mediante (Git, s.f.) y (GitHub, s.f.).
- Documentar el proceso en todas sus fases (análisis, diseño, implementación).

2.4. Alcance y limitaciones.

El alcance del proyecto incluye:

- CRUD de recetas.
- Búsqueda básica.
- Subida opcional de imágenes.
- Diseño responsive.
- Persistencia en base de datos NoSQL.

Las limitaciones iniciales son:

- No incluye autenticación de usuarios en esta primera versión.
- No se implementan notificaciones, favoritos avanzados ni exportación.

3. Análisis y planificación.

3.1. Necesidad detectada.

Muchos usuarios gestionan sus recetas de forma dispersa: notas, fotos, mensajes, webs, etc. Esto dificulta encontrar una receta concreta, mantenerla actualizada y conservar recetas familiares. Yummy! surge como respuesta a esta necesidad: centralizar, organizar y facilitar el acceso a un recetario personal digital.

3.2. Relación con proyectos tipo.

El sistema se corresponde con una aplicación CRUD de contenido, uno de los modelos más representativos del desarrollo web y multiplataforma. Este tipo de proyecto permite trabajar: diseño de modelo de datos, lógica backend, exposición de endpoints API, interfaz responsive, accesibilidad y control de versiones.

3.3. Contexto específico: accesibilidad y valor personal

El proyecto está guiado por un compromiso personal con la accesibilidad y el diseño inclusivo. Quiero que Yummy! sea fácil de usar para personas con cualquier nivel de experiencia digital, incluyendo a mis padres y mi suegra, que forman parte del origen emocional del recetario. Por ello, quiero construir la interfaz con criterios de legibilidad, orden visual y ausencia de barreras.

3.4. Requisitos funcionales.

Los requisitos funcionales describen las acciones que el usuario podrá realizar dentro de Yummy! y definen el comportamiento esperado del sistema.

- Gestión de recetas CRUD. El usuario podrá crear nuevas recetas, consultarlas, modificarlas y eliminarlas de manera sencilla.
- Añadir información completa de una receta. Cada receta podrá incluir:
    - Nombre del plato.
    - Ingredientes.
    - Pasos de preparación.
    - Categoría o etiqueta.
    - Tiempo estimado de preparación.
    - Dificultad.
    - Imagen opcional.
- Listado general de recetas. El sistema mostrará todas las recetas registradas en un panel claro y navegable.
- Búsqueda por nombre. El usuario podrá buscar recetas por texto, filtrando resultados en tiempo real o mediante un campo de búsqueda.
- Filtrado por categorías o etiquetas. Permitirá organizar las recetas por tipo.
- Visualización detallada. El usuario podrá acceder a la vista completa de una receta con toda su información.
- Subida opcional de imágenes. Permitirá asociar una fotografía del plato desde el dispositivo local.

- Diseño responsive. La interfaz permitirá visualizar las recetas tanto en ordenador como en móvil o Tablet.
- Confirmación de acciones sensibles. Al borrar o modificar una receta, el sistema mostrará mensajes de confirmación para evitar eliminaciones accidentales.
- Ordenación por fecha o nombre. El usuario podrá ordenar el recetario para facilitar su consulta.

3.5. Requisitos no funcionales.

Los requisitos no funcionales garantizan la calidad, buen funcionamiento y accesibilidad del sistema.

- Accesibilidad (criterio prioritario). La aplicación seguirá principios recomendados por las (WCAG, s.f.) tales como contraste adecuado, textos legibles, jerarquía semántica correcta, elementos interactivos claramente identificados.
- Usabilidad. La interfaz deberá ser intuitiva, con una navegación clara y botones visibles para cualquier tipo de usuario.
- Rendimiento. El tiempo de carga de las páginas deberá ser razonable, evitando esperas prolongadas.
- Portabilidad. El sistema funcionará en distintos navegadores modernos (Chrome, Firefox, Edge) y dispositivos.
- Escalabilidad. El diseño de la base de datos y la API permitirá en un futuro ampliar funcionalidades como autenticación de usuarios o favoritos.
- Mantenibilidad. El código seguirá una estructura clara, con carpetas organizadas por componentes y control de versiones mediante GitHub.
- Diseño responsive. La aplicación deberá adaptarse correctamente a diferentes tamaños de pantalla sin comprometer la legibildiad.
- Consistencia visual. Se empleará una paleta de colores coherente y una tipografía uniforme para mantener una estética clara y agradable.

3.6. Análisis de Usuarios y Roles.

En esta primera versión de Yummy! el sistema no incluye autenticación, por lo que solo existe un único perfil de usuario al que denominaré “Usuario general”. El usuario general es la persona que utiliza la aplicación para gestionar sus recetas personales. Entre sus características principales:

- Puede tener niveles de experiencia técnica muy diferentes.
- Puede incluir personas mayores, con baja visión o reducida familiaridad con apps.

- Puede ser alguien que simplemente quiere tener sus recetas ordenadas sin mayor complicación.

Los objetivos del usuario podrían ser los siguientes:

- Guardar sus recetas sin perderlas.
- Tener un recetario claro y fácil de consultar.
- Acceder a las recetas de manera intuitiva, fácil y con menús sencillos.

3.7. Casos de Uso.

Los casos de uso describen las interacciones principales entre el Usuario y el sistema Yummy!. Se agrupan en dos grandes categorías funcionales – Gestionar contenido y consultar y organizar recetario -.

- Crear receta. El usuario introduce los detalles de la receta y la guarda en la base de datos.
- Consultar receta. El usuario visualiza el listado general y selecciona una receta para ver sus detalles.
- Editar receta. El usuario modifica la información existente y guarda los cambios.
- Eliminar receta. El usuario borra una receta, previa confirmación.
- Buscar recetas. El usuario introduce un término y el sistema filtra las recetas coincidentes.
- Filtrar por categoría. El usuario selecciona una categoría para visualizar solo las recetas relacionadas con esta.
- Subir imagen. El usuario añade una fotografía del plato (opcional).
- Ordenar recetas. El usuario reorganiza la lista según el nombre o la fecha.

3.8. Diagramas UML.

Para representar de manera visual la estructura y funcionamiento he usado diferentes diagramas UML (realizados con la herramienta DIA).

- Diagrama de Casos de Uso. Muestra las acciones principales que el usuario puede realizar en la aplicación.

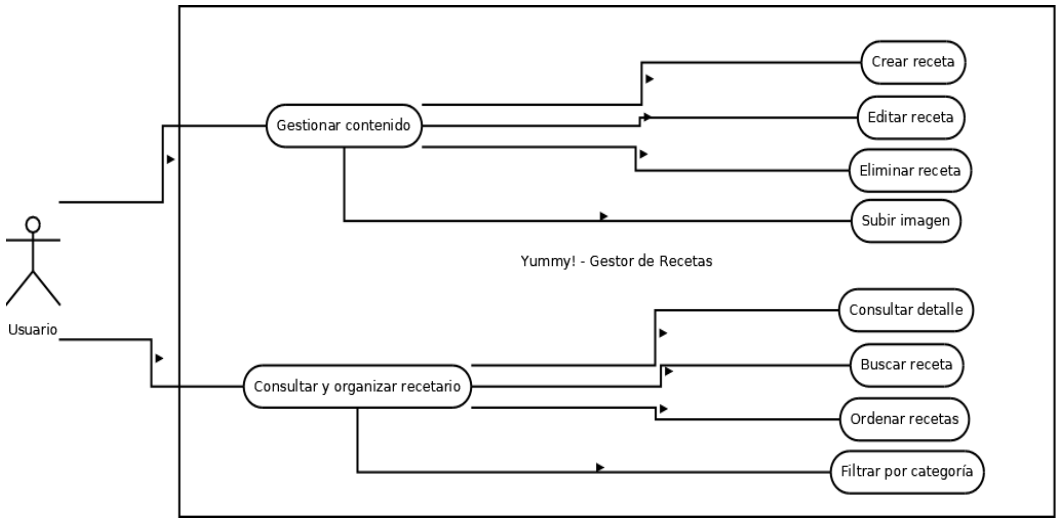

Ilustración 1. Diagrama UML de Casos de Uso.

- Diagrama de clases (versión inicial). Representa los elementos clave de la lógica interna de nuestra aplicación. Este diagrama servirá como base para el diseño backend.

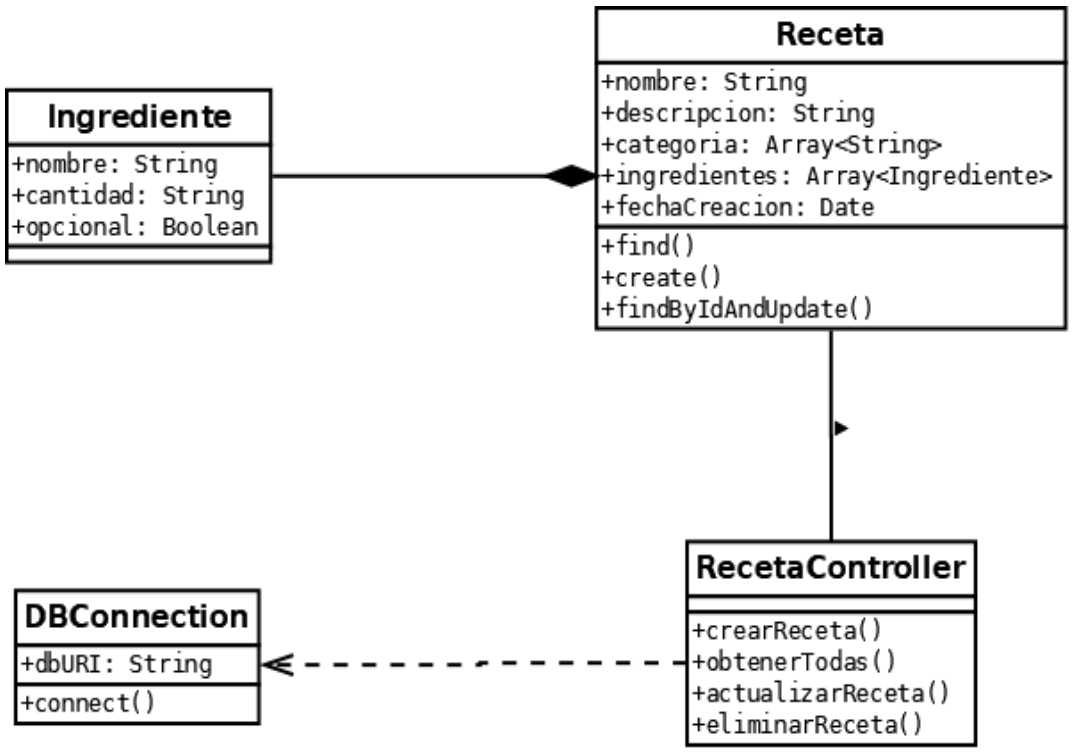

Ilustración 2. Diagrama UML de clases.

- Diagrama de secuencia. Se encargará de describir el flujo desde que el usuario realiza una acción hasta que la API procesa los datos y los guarda en la base de datos.

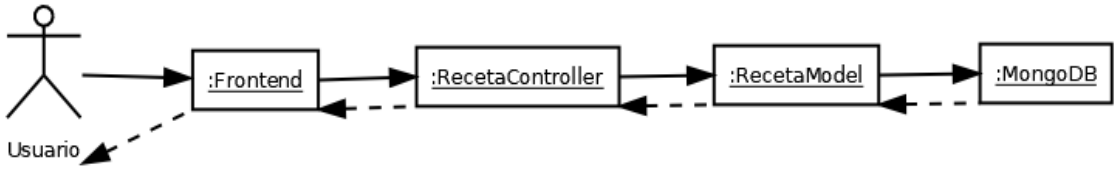

Ilustración 3. Diagrama UML de secuencia

4. Desarrollo e Implementación.

4.1. Configuración del Entorno de Desarrollo y Control de Versiones.

Antes de iniciar cualquier desarrollo de código, es fundamental establecer un entorno de trabajo organizado y profesional.

4.2. Herramientas y Entorno de Ejecución.

Para la ejecución del proyecto, he seleccionado las siguientes herramientas:

- IDE (Entorno de Desarrollo): He seleccionado Visual Studio Code por su ligereza y su excelente soporte para el desarrollo full-stack basado en JavaScript.
- Lenguaje de Programación: El proyecto se ejecutará en Node.js para el backend y gestión de la API. La verificación de las versiones instaladas (Node.js v22.10.0 y npm 10.8.82) se realizó con éxito antes de la inicialización.

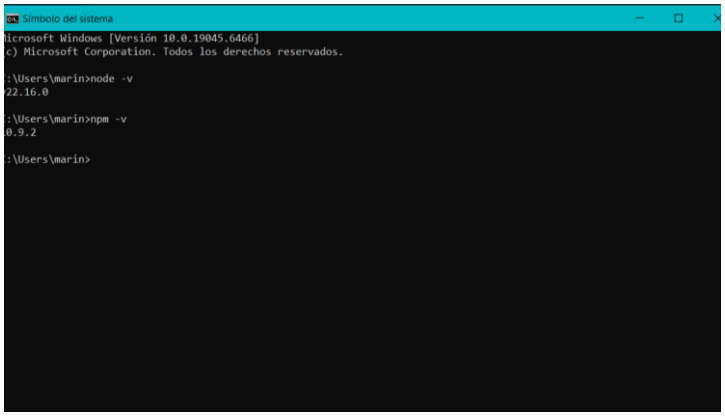

Ilustración 4. Verificación de versiones instaladas de Node.js.

- Control de versiones: Utilizaré Git junto con GitHub para la gestión del repositorio y el control riguroso de versiones, tal y como se define en la metodología.

4.3. Gestión del Repositorio en GitHub.

Para garantizar el control del desarrollo, creé el repositorio remoto “Yummy-Gestor-Recetas” en GitHub con la descripción “Proyecto Fin de Grado: Aplicación full-stack para la gestión de recetas personales”.

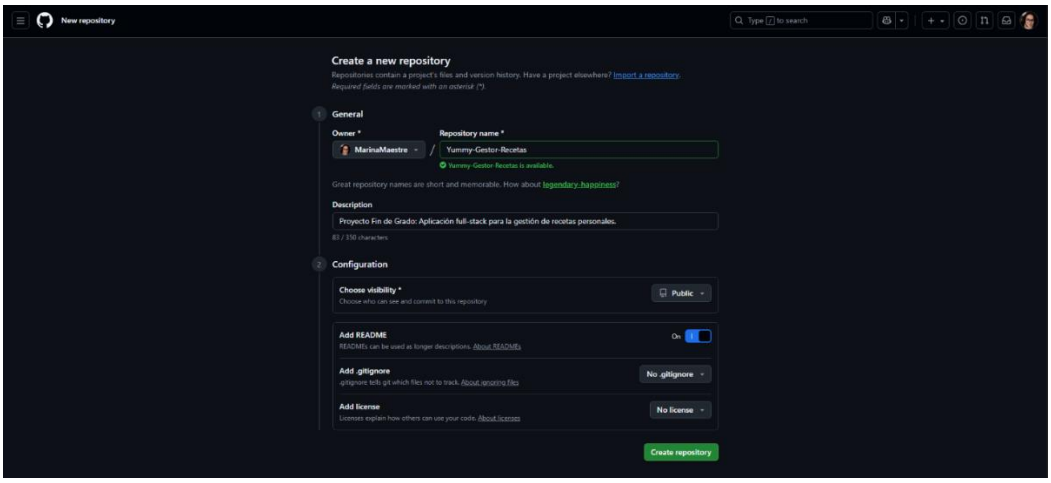

Ilustración 5. Creación repositorio en GitHub.

Luego, cloné el repositorio a mi máquina local. De este modo creé una copia de trabajo local que automáticamente configura el enlace con el repositorio de GitHub. Utilizaré esta copia clonada como punto de partida para comenzar a escribir código. A partir de este momento, puedo utilizar los comandos *git commit* para registrar los cambios en mi máquina local y *git push* para subir mi progreso a la nube, asegurando el control riguroso de versiones que definí como objetivo en mi proyecto.

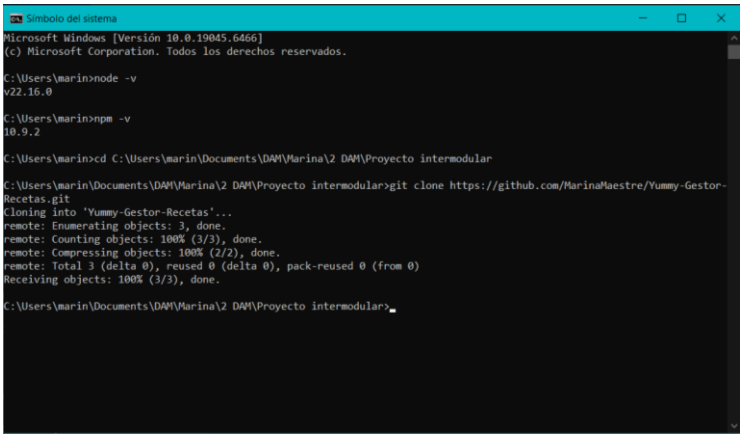

Ilustración 6. Clonación de repositorio.

4.4. Estructura de Directorios e Inicialización.

Una vez clonado el repositorio, el primer paso de la fase de implementación fue organizar una estructura de directorios clara y coherente con la arquitectura full-stack planteada durante el diseño. Para ello, creé tres carpetas raíz que separan de forma limpia cada parte del proyecto: backend, donde

se encuentra toda la lógica del servidor; frontend, que contendrá la interfaz desarrollada por React; y docs, destinada a la documentación del proyecto, incluyendo esta memoria y los diagramas UML.

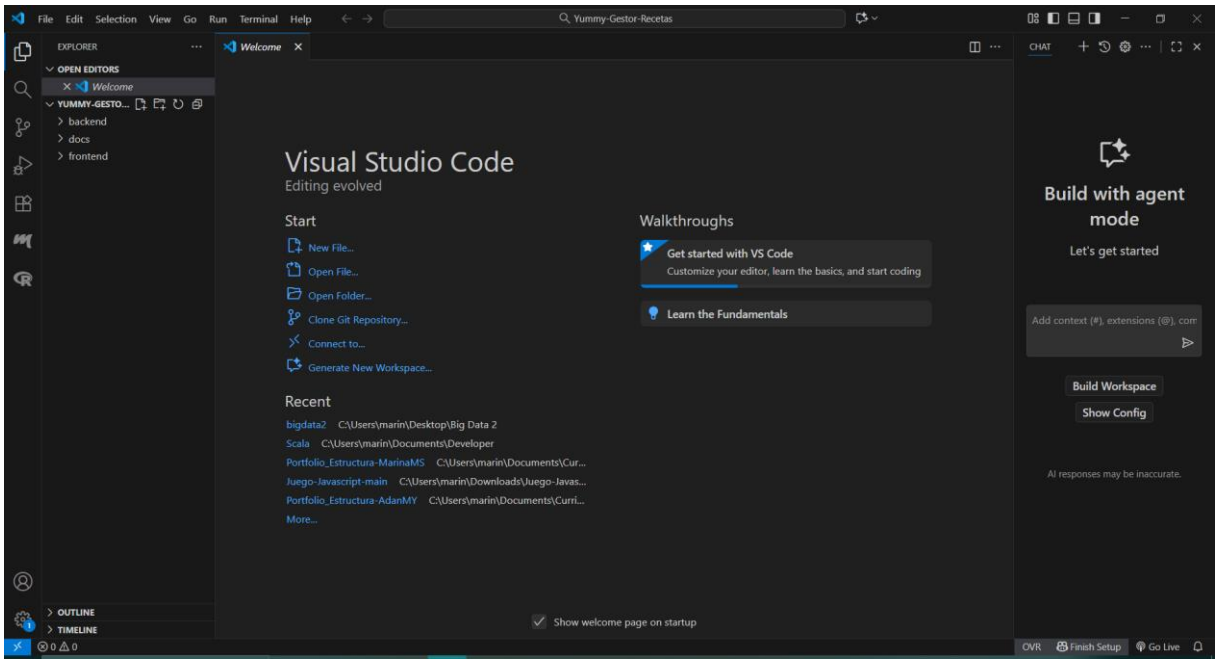

Ilustración 7. Estructura inicial de carpetas en Visual Studio Code.

Con la estructura base definida, pasé a configurar el entorno del backend. Dentro de la carpeta backend ejecuté npm init -y, lo que generó el archivo packaje.json. Este archivo es fundamental, ya que gestiona todas las dependecias, scripts y metadatos del servidor.

Después instalé los módulos principales que necesito para construir la API:

- Express, el framework que utilizaré para definir y gestionar los endpoints de API REST.
- Mongoose, que me permitirá modelar los datos de las recetas y facilitará la comunicación con la base de datos MongoDB.
- Dotenv, imprescindible para manejar variables de entorno como la cadena de conexión a la base de datos, manteniendo así cualquier datos sensible fuera del código público del proyecto.

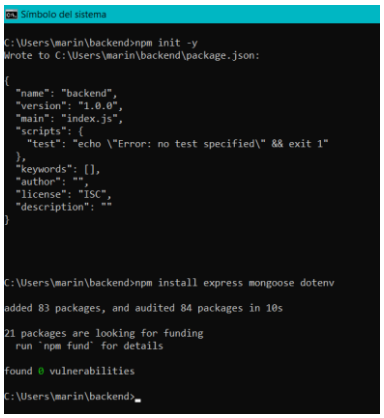

Ilustración 8. Símbolo de sistema con la primera fase de la implementación.

Con estas dependencias ya instaladas y configuradas, el servidor está preparado para dar el siguiente paso: establecer la conexión con MongoDB y comenzar a implementar la estructura real de la API. Esto marca el inicio práctico de la construcción de Yummy! desde el backend.

4.5. Configuración del Backend y Dependencias.

Una vez establecí el servidor base, procedí a dotar al proyecto de las herramientas necesarias para cumplir con los requisitos de seguridad exigidos. Al inspeccionar el archivo package.json, puedo confirmar que la instalación de las dependencias críticas se ha realizado con éxito.

Específicamente, he integrado Bcryptjs para asegurar el tratamiento de datos sensibles mediante el cifrado de contraseñas, y Jsonwebtoken para la futura implementación del sistema de autenticación de usuarios. Estas librerías, junto con Express y Mongoose, constituyen el núcleo técnico de mi aplicación.

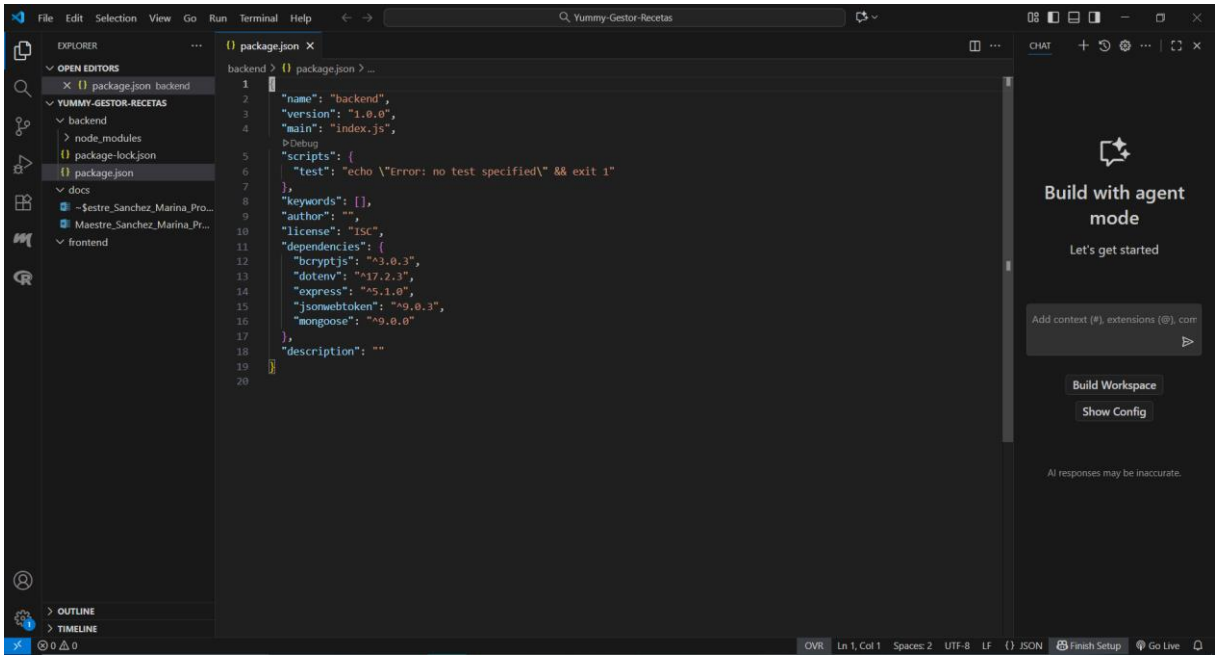

Ilustración 9. Detalle de las dependencias críticas de seguridad y funcionamiento en el archivo package.json.

4.6. Configuración de Variables de Entorno.

Con el objetivo de seguir las buenas prácticas en el desarrollo de APIs y proteger la información sensible, he implementado un archivo de configuración .env en la raíz del servidor. En este archivo he centralizado datos críticos como el puerto de ejecución, la cadena de conexión a la base de datos MongoDB y la clave secreta para la generación de tokens JWT. Esta técnica garantiza que las credenciales no queden expuestas en el código fuente público, facilitando además la portabilidad del sistema.

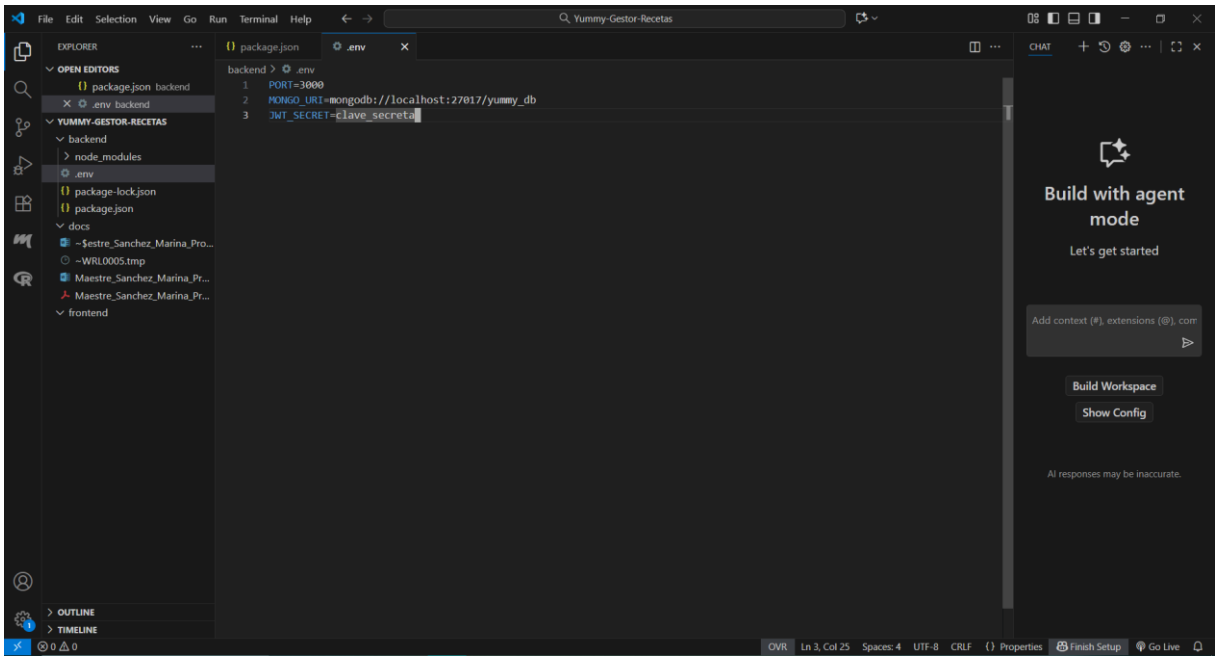

Ilustración 10. Configuración de variables de entorno en el archivo .env.

4.7. Arquitectura del Backend y Configuración del Servidor Express.

Para poner en marcha el corazón de la aplicación, he desarrollado el archivo server.js, que actúa como el punto de entrada principal del sistema. Este script se encarga de inicializar el framework Express, cargar las variables de entorno mediante la librería dotenv y habilitar el procesamiento de datos en formato JSON.

Tras asegurar el correcto guardado del código y situarme en el directorio /backend mediante la terminal, ejecuté el comando node server.js. Como se observa en la siguiente ilustración, he verificado que el servidor levanta exitosamente, confirmando la inyección de las variables de entorno y quedando a la escucha de peticiones en el puerto 3000.

```
const express = require('express');
const dotenv = require('dotenv');

// Cargo las variables de entorno
dotenv.config();

const app = express();

// Middleware para procesar JSON
app.use(express.json());

// Ruta de prueba inicial
app.get('/', (req, res) => {
    res.send('🚀 Servidor de Yummy! funcionando correctamente');
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`✅ Servidor ejecutándose en el puerto ${PORT}`);
});
```

```
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas>
cd backend
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas\backend> node server.js
[dotenv@17.2.3] injecting env (3) from .env -- tip: ⚙ specify custom .env file path with { path: '/custom/path/.env' }
✅ Servidor ejecutándose en el puerto 3000
```

Ilustración 11. Ejecución exitosa del servidor Express y validación de variables de entorno.

4.8. Módulo de Conexión a la Base de Datos

Una vez que logré arrancar el servidor básico, el siguiente paso fue establecer el vínculo con la base de datos para que el recetario tenga memoria. Para ello, he creado un módulo específico dentro de una nueva carpeta llamada “config”.

```
const mongoose = require('mongoose');

const conectarDB = async () => {
    try {
        await mongoose.connect(process.env.MONGO_URI);
        console.log('✅ Conexión exitosa a MongoDB');
    } catch (error) {
        console.error('❌ Error de conexión:', error.message);
        process.exit(1);
    }
};

module.exports = conectarDB;
```

```
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas>
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas>
```

Ilustración 12. Código de db.js.

Para la gestión del almacenamiento NoSQL, he seleccionado MongoDB Community Server como motor de base de datos. Tras realizar la instalación y configuración del servicio en mi entorno

local, he vinculado la aplicación mediante la librería Mongoose. He verificado la correcta integración a través de la consola de depuración, confirmando que el servidor es capaz de establecer una conexión estable y realizar operaciones sobre la base de datos de manera asíncrona.

```
dotenv.config();

const app = express();

app.use(express.json());

app.get('/', (req, res) => {
    res.send('🚀 Servidor de Yummy! funcionando correctamente');
});

const PORT = process.env.PORT || 3000;

// Primero conectamos a la DB y luego arrancamos el servidor
conectarDB().then(() => {
    app.listen(PORT, () => {
        console.log(`✅ Servidor ejecutándose en el puerto ${PORT}`);
    });
});
```

```
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas> cd backend
PS C:\Users\marin\Documents\DAM\Marina\2 DAM\Yummy-Gestor-Recetas\backend> node server.js
[dotenv@17.2.3] injecting env (3) from .env -- tip: 🔐 encrypt with Dotenvx: https://dotenvx.com
✅ Conexión exitosa a MongoDB
✅ Servidor ejecutándose en el puerto 3000
```

Ilustración 13. Validación final de conexión exitosa entre el servidor y MongoDB

4.9. Lógica de autenticación y cifrado

Para garantizar la seguridad de los usuarios, he implementado un sistema de registro que no solo almacena datos, sino que los protege. He configurado el controlador authController.js para que, antes de guardar cualquier información en MongoDB, la contraseña pase por un proceso de hashing con la librería Bcryptjs. Como se detalla en el código del modelo Usuario.js, he establecido que el email sea un campo único (unique: true), evitando duplicidades en la base de datos y reforzando la integridad del sistema.

```
const mongoose = require('mongoose');

const usuarioSchema = mongoose.Schema({
    nombre: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true }
}, { timestamps: true });

module.exports = mongoose.model('Usuario', usuarioSchema);
```

Ilustración 14. Código Usuario.js

Para asegurar que solo los usuarios registrados puedan gestionar recetas, he implementado un middleware de autenticación en el archivo auth.js. Este componente actúa como un filtro de seguridad que intercepta las peticiones, verifica la validez del token JWT y, en caso de éxito, permite el acceso a las funciones del backend. Con esta pieza, la arquitectura del sistema queda blindada contra accesos no autorizados.

```
const jwt = require('jsonwebtoken');

module.exports = function(req, res, next) {
    // Leer el token del header
    const token = req.header('x-auth-token') || req.header('Authorization')?.replace('Bearer ', '');

    // Revisar si no hay token
    if (!token) {
        return res.status(401).json({ msg: 'No hay token, permiso no válido' });
    }

    // Validar el token
    try {
        const cifrado = jwt.verify(token, process.env.JWT_SECRET);
        req.usuario = cifrado.usuario;
        next();
    } catch (error) {
        res.status(401).json({ msg: 'Token no válido' });
    }
};
```

Ilustración 15. Código middleware de autenticación.

4.10. Validación del Sistema con Postman.

Para revisar que todo lo que he programado funciona de verdad, he usado una herramienta llamada Postman. Como todavía no tengo hecha la parte visual de la web, Postman me sirve para

mandarle los datos (nombre, email y contraseña) directamente a mi servidor para ver si responde.

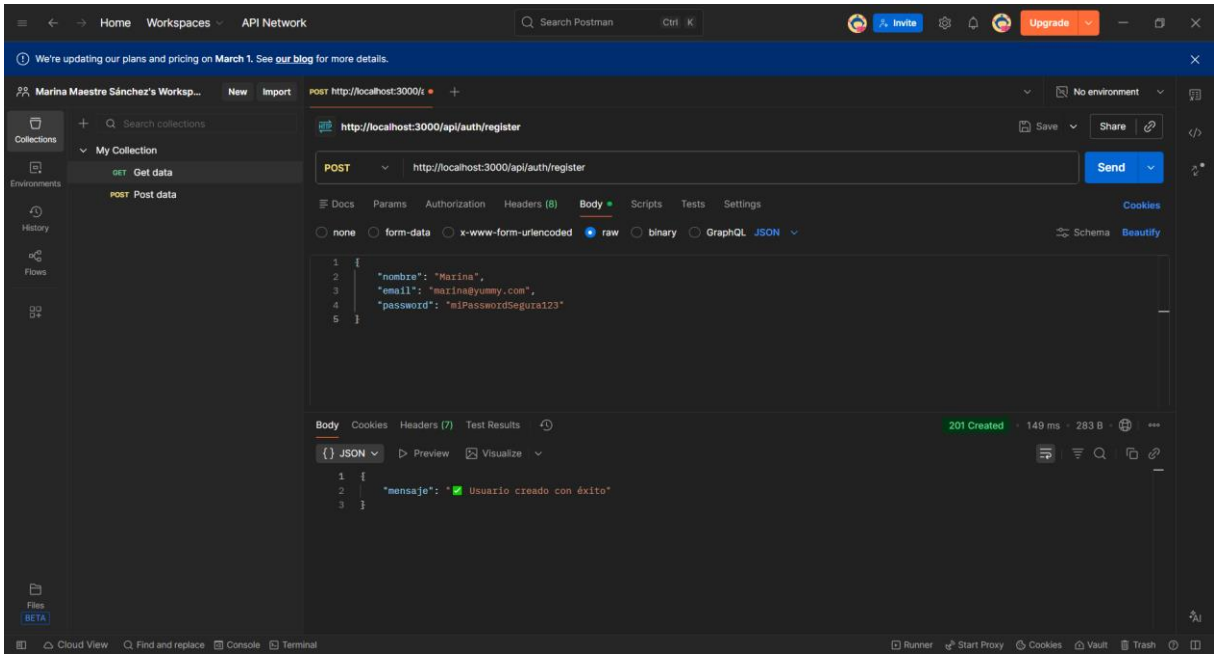

Ilustración 16. Creación de primer usuario desde Postman

Como se ve en la ilustración, al darle al botón de enviar, el sistema me ha devuelto un mensaje de éxito: 'Usuario creado con éxito'. Esto es una victoria para mí porque significa que mi servidor ha recibido bien los datos.

4.11. Modelado de datos para el sistema de recetas.

Una vez que he comprobado que el sistema de usuarios funciona, el siguiente paso ha sido diseñar el 'molde' de lo que realmente importa en Yummy!: las recetas . Para que la aplicación sea útil y accesible para mi familia, no me bastaba con un simple campo de texto; necesitaba una estructura clara.

Como se puede ver en el código del archivo Receta.js, he definido campos específicos para el título, la descripción, una lista de ingredientes y los pasos a seguir . Un detalle técnico muy importante es la relación que he creado entre el usuario y la receta mediante el campo ref: 'Usuario'. Gracias a esto, el sistema sabrá siempre quién ha escrito cada receta, permitiendo que cualquier usuario tenga su propio espacio privado y organizado dentro de la aplicación.

```
const mongoose = require('mongoose');

const recetaSchema = mongoose.Schema({
    titulo: { type: String, required: true },
    descripcion: { type: String },
    ingredientes: [{
        nombre: { type: String, required: true },
        cantidad: { type: String }
    }],
    pasos: [{ type: String, required: true }],
    dificultad: { type: String, enum: ['Fácil', 'Media', 'Difícil'], default: 'Media' },
    tiempo: { type: Number }, // En minutos
    usuario: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
    foto: { type: String } // URL de la imagen
}, { timestamps: true });

module.exports = mongoose.model('Receta', recetaSchema);
```

Ilustración 17. Código del modelo de datos para recetas.

4.12. Lógica de gestión de recetas.

Con el molde ya listo, el siguiente paso ha sido crear el 'cerebro' que gestiona las recetas: el controlador. He desarrollado el archivo recetaController.js para que el sistema sepa qué hacer cuando reciba una nueva delicia culinaria.

Lo que más me gusta de esta parte es que he conseguido que el servidor extraiga automáticamente el ID del usuario. De esta forma, cuando por ejemplo mi madre guarde sus lentejas, el sistema sabrá que son suyas y las dejará a buen recaudo bajo su perfil. He incluido también un sistema de control de errores para que, si algo falla al guardar, el sistema nos avise y no perdamos la información.

```
const Receta = require('../models/Receta');

// Crear una nueva receta
exports.crearReceta = async (req, res) => {
    try {
        const { titulo, descripcion, ingredientes, pasos, dificultad, tiempo } = req.body;

        const nuevaReceta = new Receta({
            titulo,
            descripcion,
            ingredientes,
            pasos,
            dificultad,
            tiempo,
            usuario: req.usuario.id // Esto lo vincula al usuario que está logueado
        });

        await nuevaReceta.save();
        res.status(201).json({ msg: '¡Receta guardada con éxito!', nuevaReceta });
    } catch (error) {
        res.status(500).send('Hubo un error al guardar la receta');
    }
};
```

Ilustración 18. Código del controlador de recetas.

4.13. Definición de rutas y protección de recursos.

Para que el servidor sepa dirigir las peticiones hacia la lógica que he programado, he creado el archivo recetaRoutes.js. En este punto, la seguridad ha sido mi prioridad: no quiero que cualquiera pueda añadir recetas, sino solo los usuarios registrados. Por eso, he implementado el middleware auth como un 'guardián' en la ruta. Esto significa que el sistema primero verifica que el usuario es quien dice ser y, solo entonces, le permite acceder al controlador para guardar su receta. Es un paso fundamental para que el recetario familiar sea un entorno seguro y privado.

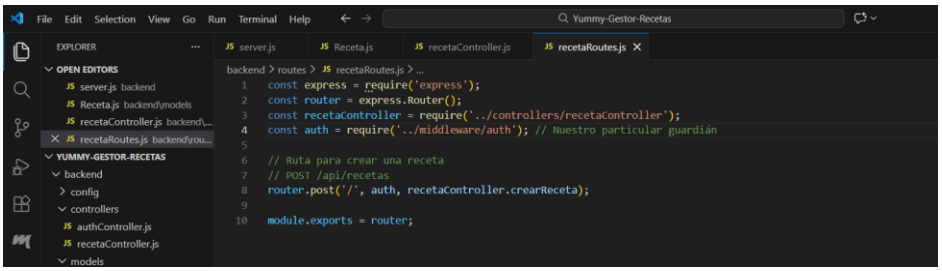

Ilustración 19. Configuración de rutas para el módulo de recetas.

4.14. Validación final del sistema de autenticación y seguridad

Tras una fase intensiva de depuración de rutas y persistencia en la inicialización del servidor he validado con éxito el funcionamiento del módulo de seguridad mediante Postman. Como se observa en la Ilustración siguiente, al enviar las credenciales de usuario a la ruta /api/auth/login, el servidor responde correctamente.

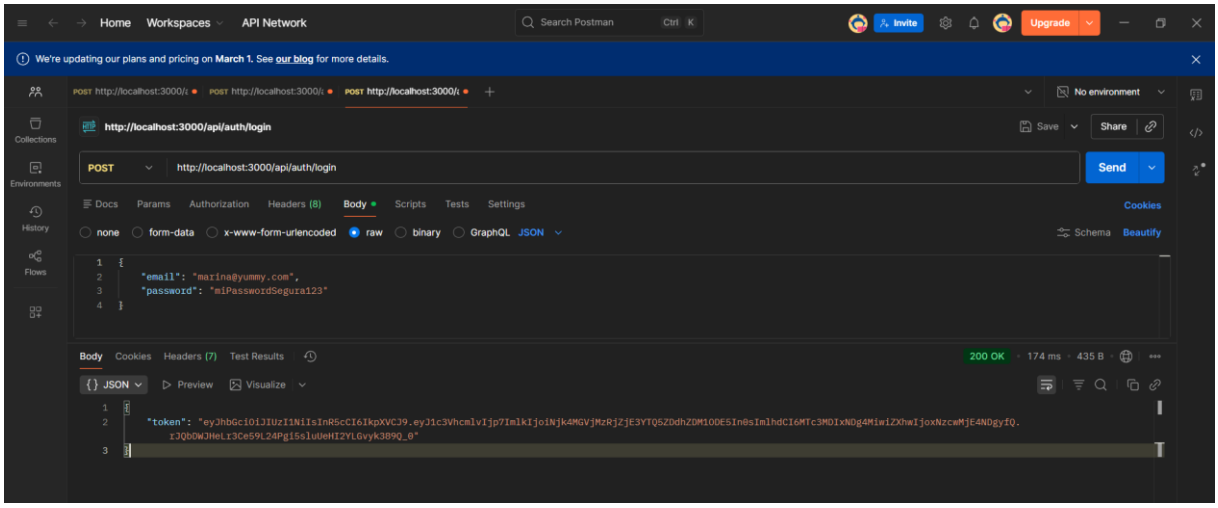

Ilustración 20. Generación exitosa de JSON Web Token.

Este proceso confirma que la arquitectura backend de Yummy! no solo se conecta correctamente a la base de datos MongoDB, sino que también implementa un sistema de cifrado y

autorización robusto, garantizando que el recetario familiar sea un entorno privado y seguro para el usuario.

4.15. Conclusión de la arquitectura backend y éxito de persistencia.

Tras una exhaustiva fase de depuración técnica, he validado con éxito el flujo completo de la aplicación Yummy!. Se ha verificado que el servidor procesa correctamente objetos JSON complejos con estructuras anidadas (ingredientes con nombre y cantidad), garantizando la integridad de los datos según el modelo definido en Mongoose. La obtención del código de estado 201 Created confirma que la API es capaz de recibir, autenticar y almacenar información de forma persistente en el clúster de MongoDB, dejando la infraestructura backend totalmente operativa y lista para dar soporte a la interfaz de usuario en la siguiente fase de desarrollo.

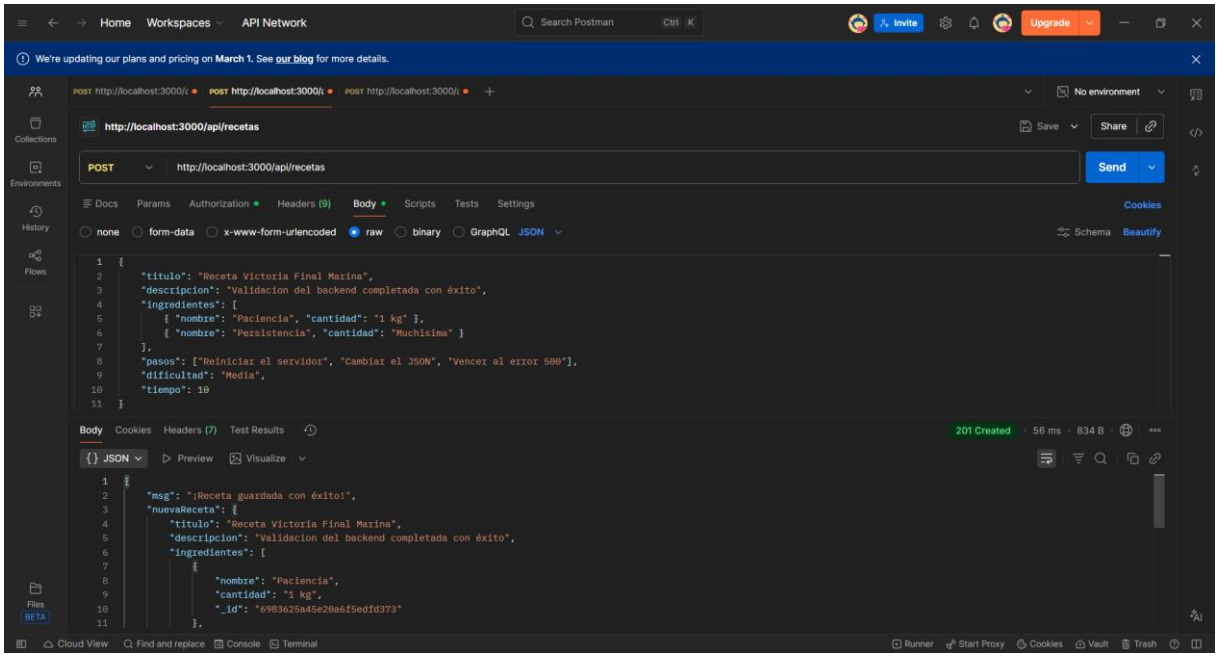

Ilustración 21. Confirmación final de guardado de receta en MongoDB.

5. Bibliografía y Webgrafía.

Bcryptjs. (s.f.). *Optimized bcrypt in JavaScript con zero dependencies*. Obtenido de https://www.npmjs.com/package/bcryptjs

Express.js. (s.f.). *Guía oficial de direccionamiento y middleware para Node.js*. Obtenido de https://expressjs.com/es/

Express.js. (s.f.). *Router: Creación de manejadores de rutas modulares y montables*. Obtenido de https://expressjs.com/es/guide/routing.html

GitHub. (s.f.). *Documentación oficial para la gestión de repositorios y control de versiones*. Obtenido de https://docs.github.com/es

Microsoft. (s.f.). *Visual Studio Code: Documentación oficial del editor de código fuente*. Obtenido de https://code.visualstudio.com/docs

MongoDB. (s.f.). *Manual de instalación y configuración de MongoDB Community Server*. Obtenido de https://www.mongodb.com/docs/manual/

Mongoose. (s.f.). *Documentación oficial de modelos y esquemas para Node.js*. Obtenido de https://mongoosejs.com/

Node.js Foundation. (s.f.). *Documentación técnica del entorno de ejecución de JavaScript*. Obtenido de https://nodejs.org/es/

Postman. (s.f.). *Manual de pruebas para servicios web y APIs*. Obtenido de https://learning.postman.com/

W3C. (2025). *Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1*. Obtenido de https://www.w3.org/WAI/standards-guidelines/wcag/es/